

LAB FILE
Of
“PE/UE Lab, AIML-803”
(AIML)



Submitted in partial fulfilment of the requirement for the
award of the degree of
BACHELOR OF TECHNOLOGY
In
COMPUTER SCIENCE AND ENGINEERING – AIML

Submitted By
STUDENT NAME: Palak Janbhor
(ENROLLMENT NO. LNCCBTC11149)

Under the supervision of:
Prof. Gaurav Saxena

Forwarded By:
Dr. Devdas Saraswat
HOD CSE, AIML

CONTENT

S. No.	Experiment Name	Experiment Date	Submission Date	Signature
1.	Illustrate various hardware components with the help of IoT Kit.			
2.	Illustrate essential software components of IoT.			
3.	Design prototype of IoT service oriented architecture.			
4.	Illustrate with kit Wireless IoT sensors.			
5.	Study and analyse Arduino IoT platform and design prototype.			
6.	Develop a simplified version of basic cryptocurrency prototype using a blockchain development platform.			
7.	Create simplified version of a Bitcoin wallet using software like Electrum.			
8.	Introduce the simplified version of Lamport -Shostak-Pease BFT algorithm for achieving Byzantine fault tolerance.			
9.	Implement a simplified version of blockchain solution for facilitating and tracking trade transactions.			
10	Implementing a simplified version of Ripple involves creating a basic blockchain with a consensus algorithm.			

EXPERIMENT 01

Q.1 Illustrate various hardware components with the help of IoT Kit.

ANS: Aim -

To study and illustrate different hardware components used in an IoT Kit and understand their functions.

Introduction:

Internet of Things (IoT) is a network of physical devices connected through the internet that collect and exchange data. An IoT kit contains different hardware components used for building smart systems such as smart homes, smart agriculture, healthcare devices, etc. These components help in sensing, processing, communication, and controlling operations.

Hardware Components of IoT Kit:

1. Microcontroller Board (Arduino Uno / NodeMCU)

- It is the main processing unit of the IoT kit.
- It reads data from sensors and controls output devices.
- Arduino Uno uses ATmega328P microcontroller.
- NodeMCU has built-in Wi-Fi module.

Functions:

- Processes input data
- Controls sensors and actuators
- Executes programmed instructions

Diagram:

```
-----  
| Arduino Uno |  
| Digital Pins |  
| Analog Pins |  
| USB Port    |  
| Power Port   |  
-----
```

2. Breadboard

- Used for temporary circuit connections without soldering.
- Helps in testing circuits easily.

Functions:

- Connects electronic components
- Makes prototyping simple

Diagram:

```
-----  
| o o o o o o o o o o |  
| o o o o o o o o o o |  
-----
```

```

|-----|
| 0 0 0 0 0 0 0 0 0 0 |
| 0 0 0 0 0 0 0 0 0 0 |
|-----|

```

3. Temperature Sensor (LM35 / DHT11)

- Measures temperature.
- DHT11 can also measure humidity.

Functions:

- Detects environmental temperature
- Sends data to microcontroller

Diagram:



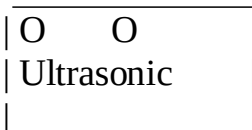
4. Ultrasonic Sensor

- Measures distance using sound waves.

Functions:

- Used in obstacle detection
- Water level monitoring

Diagram:



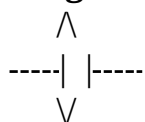
5. LED (Light Emitting Diode)

- Output device that glows when current passes.

Functions:

- Indicates ON/OFF condition
- Used in smart lighting systems

Diagram:



6. Relay Module

- Used to control high voltage devices.

Functions:

- Switches appliances ON/OFF
- Used in home automation

Diagram:

| Relay Module |
IN VCC GN

7. Wi-Fi Module (ESP8266)

- Provides internet connectivity to IoT devices.

Functions:

- Sends sensor data to cloud
- Enables remote monitoring

Diagram:

ESP8266
Wi-Fi

8. LCD Display

- Displays output information.

Functions:

- Shows sensor values
- Displays messages

Diagram:

| LCD DISPLAY |
Temperature: 30C

Working Principle:

1. Sensors collect data from environment.
2. Microcontroller processes the data.
3. Communication module sends data to internet/cloud.
4. Output devices perform required actions.

Applications:

- Smart Home Automation
- Smart Agriculture
- Health Monitoring
- Smart Traffic System
- Industrial Automation

EXPERIMENT 02

Q1. Illustrate essential software components of IoT.

ANS.

Aim -

To study and illustrate the essential software components used in Internet of Things (IoT) systems.

Introduction:

IoT software components are responsible for controlling hardware devices, processing data, enabling communication, and providing user interaction. These software elements help IoT devices connect with each other and perform smart operations efficiently.

Essential Software Components of IoT:

1. Operating System (OS)

An operating system manages hardware resources and allows software applications to run on IoT devices.

Examples:

- Linux
- FreeRTOS
- Windows IoT

Functions:

- Memory management
- Process scheduling
- Device management

Diagram:

```
+-----+
| Operating System |
+-----+
```

2. Programming Languages

Used to write code for IoT applications and device control.

Examples:

- Python
- Java
- C++

Functions:

- Develop IoT applications
- Control sensors and devices

Diagram:

```
+-----+
| Programming Language |
+-----+
```

3. Embedded Software / Firmware

Firmware is stored inside IoT devices and controls hardware operations.

Functions:

- Reads sensor data
- Controls actuators
- Performs device operations

Diagram:

```
+-----+  
| Firmware |  
+-----+
```

4. Cloud Platform

Stores and processes IoT data remotely.

Examples:

- Amazon Web Services IoT
- Google
- Microsoft

Functions:

- Data storage
- Data analysis
- Remote monitoring

Diagram:

Device ---> Cloud ---> User

5. Database Management System

Stores large amounts of IoT-generated data.

Examples:

- MySQL
- MongoDB

Functions:

- Store records
- Manage data retrieval

Diagram:

```
+-----+  
| Database |  
+-----+
```

6. Network Protocol Software

Enables communication between IoT devices.

Examples:

- MQTT
- HTTP
- CoAP

Functions:

- Data transmission
- Device communication

Diagram:

Device <----> Internet <----> Server

7. Mobile/Web Applications

Used by users to monitor and control IoT devices.

Examples:

- Blynk
- Web dashboards

Functions:

- Remote control
- Data visualization

Diagram:

User ---> Mobile App ---> IoT Device

Working of IoT Software Components:

1. Firmware collects data from sensors.
2. OS manages device operations.
3. Network protocols transfer data.
4. Cloud stores and processes data.
5. Applications display information to users.

Applications:

- Smart homes
- Healthcare monitoring
- Smart cities
- Industrial automation
- Agriculture systems

EXPERIMENT 03

Q1. Design prototype of IoT service oriented architecture.

ANS -

Aim:

To design and study the prototype of IoT Service Oriented Architecture (SOA).

Introduction:

IoT Service Oriented Architecture (SOA) is a framework where different IoT devices and services communicate with each other through the internet. It allows devices, sensors, applications, and users to interact efficiently by providing services in a modular way. In IoT SOA, each component performs a specific function such as sensing, processing, communication, and service delivery.

Layers of IoT Service Oriented Architecture:

1. Sensing Layer

- Contains sensors and actuators.
- Collects data from the physical environment.
- Examples: Temperature sensor, humidity sensor, motion sensor.

Diagram:

```
+-----+  
| Sensors/Actuators |  
+-----+
```

2. Network Layer

- Transfers collected data from sensors to processing systems.
- Uses communication technologies like Wi-Fi, Bluetooth, Zigbee, GSM.

Diagram:

```
+-----+  
| Network Layer    |  
| Wi-Fi/Bluetooth  |  
+-----+
```

3. Processing Layer (Middleware Layer)

- Processes and stores data.
- Provides service management.
- Includes cloud computing and databases.

Diagram:

```
+-----+  
| Cloud/Database   |  
| Processing Layer |  
+-----+
```

4. Application Layer

- Provides services to users.
- Examples: Smart home app, healthcare app.

Diagram:

```
+-----+  
| User Application |  
+-----+
```

5. Business Layer

- Manages overall IoT business model.
- Analyzes data for decision-making.

Diagram:

```
+-----+  
| Business Layer |  
+-----+
```

Prototype Design Flow:

Sensors → Gateway → Cloud Server → Application → User

Example:

Temperature Sensor → Wi-Fi Module → Cloud Platform → Mobile App → User receives temperature data.

Components Required:

- Sensors
- Microcontroller (Arduino Uno / NodeMCU)
- Internet connection
- Cloud storage
- Mobile/Web application

Working:

1. Sensors collect real-time data.
2. Network transfers data to cloud servers.
3. Cloud processes and stores data.
4. Application provides services to users.
5. Business layer analyzes performance.

Applications:

- Smart homes
- Smart healthcare
- Smart agriculture
- Smart transportation
- Industrial IoT

Advantages:

- Easy integration of devices.

EXPERIMENT 04

Q1. Illustrate with kit Wireless IoT sensors.

ANS.

Aim:

To study and illustrate different wireless IoT sensors used in an IoT kit.

Introduction:

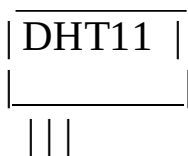
Wireless IoT sensors are smart devices that collect environmental data and transmit it wirelessly to other devices or cloud platforms using communication technologies such as Wi-Fi, Bluetooth, Zigbee, GSM, or LoRa. These sensors are widely used in smart homes, healthcare, agriculture, and industrial automation.

Wireless IoT Sensors Used in Kit:

1. Temperature and Humidity Sensor (DHT11/DHT22)

- Measures temperature and humidity.
- Sends data to microcontroller wirelessly through Wi-Fi modules.

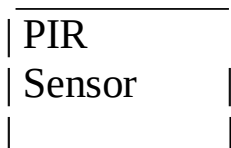
Diagram:



2. Motion Sensor (PIR Sensor)

- Detects movement of objects or humans.
- Used in smart security systems.

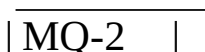
Diagram:



3. Gas Sensor (MQ-2)

- Detects smoke, LPG, and harmful gases.
- Sends alert data wirelessly.

Diagram:



| Sensor |
| |

4. Ultrasonic Sensor

- Measures distance using ultrasonic waves.
- Used in obstacle detection systems.

Diagram:

| O O |
| Ultrasonic |
| |

5. Soil Moisture Sensor

- Measures moisture level in soil.
- Used in smart irrigation systems.

Diagram:

| Soil Sensor |
| |

6. Light Sensor (LDR)

- Detects light intensity.
- Used in automatic street lights.

Diagram:

---/V---
LDR

Wireless Communication Modules:

These sensors connect wirelessly using:

- Wi-Fi module (ESP8266)
- Bluetooth Module HC-05
- Zigbee module
- GSM module
- LoRa module

Working Process:

Sensor → Microcontroller → Wireless Module → Cloud/App → User

Example:

Temperature Sensor → NodeMCU → Wi-Fi → Mobile App

Applications:

- Smart Home Automation
- Smart Agriculture
- Healthcare Monitoring
- Security Systems
- Smart Cities

Advantages:

- Remote monitoring
- Low power consumption
- Easy installation
- Real-time data transfer

Conclusion:

Thus, various wireless IoT sensors were studied and illustrated successfully using the IoT kit. These sensors play an important role in modern IoT applications.

EXPERIMENT 05

Q1. Study and analyse Arduino IoT platform and design prototype.

ANS.

Aim:

To study the Arduino IoT platform and design a basic prototype using Arduino.

Introduction:

Arduino is an open-source electronics platform based on easy-to-use hardware and software. It is widely used in IoT applications for connecting sensors, actuators, and internet services.

The Arduino IoT platform helps users build smart devices that can collect data and communicate over the internet.

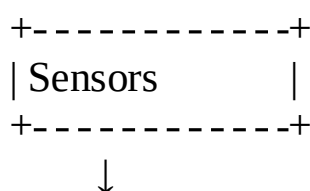
Components Required:

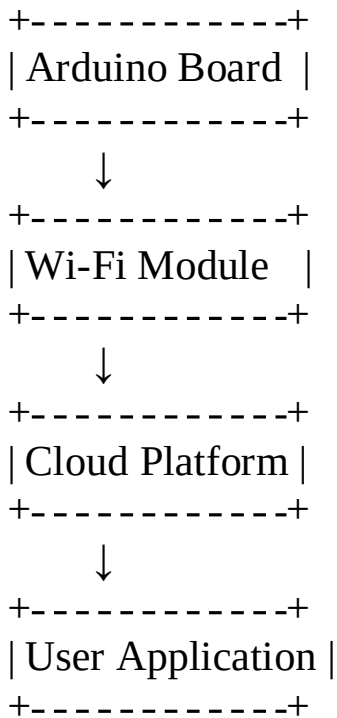
- Arduino Uno / NodeMCU
- Temperature Sensor (DHT11)
- LED
- Breadboard
- Jumper Wires
- Wi-Fi Module (ESP8266 if using Arduino Uno)
- USB Cable
- Computer/Laptop with Arduino IDE

Arduino IoT Platform Features:

1. Open-source hardware and software
2. Easy programming using Arduino IDE
3. Supports Wi-Fi and cloud connectivity
4. Compatible with many sensors
5. Low cost and user-friendly

Architecture of Arduino IoT Platform:





Prototype Design:

Smart Temperature Monitoring System

This prototype measures room temperature and sends data to the cloud.

Working:

1. DHT11 sensor collects temperature data.
2. Arduino Uno processes the data.
3. Wi-Fi module sends data to cloud platform.
4. User checks temperature on mobile application.
5. LED glows when temperature exceeds a limit.

Circuit Diagram:

DHT11 Sensor ----> Arduino
LED -----> Arduino
Wi-Fi Module ----> Arduino
Arduino -----> Cloud
Cloud -----> Mobile App

Applications:

- Smart home automation
- Weather monitoring
- Smart agriculture
- Healthcare monitoring.

EXPERIMENT 06

Q1. Develop a simplified version of basic cryptocurrency prototype using a blockchain development platform.

ANS.

Aim:

To develop a simplified prototype of a basic cryptocurrency using a blockchain development platform.

Introduction:

Cryptocurrency is a digital currency that uses blockchain technology for secure transactions. Blockchain stores transaction records in blocks that are linked together using cryptographic hashes. A basic cryptocurrency prototype helps in understanding how digital coins are created, transferred, and validated.

Popular blockchain development platforms include Ethereum, Hyperledger, and Binance Smart Chain.

Requirements:

- Computer/Laptop
- Internet connection
- Python / JavaScript
- Ganache (for local blockchain testing)
- Remix IDE or Visual Studio Code
- MetaMask (optional)

Basic Components of Cryptocurrency Prototype:

1. Users/Wallets
2. Digital Coins

3. Transactions
4. Blockchain Ledger
5. Mining/Validation

Working Flow Diagram:

User A ---- Sends Coin ---- > User B



Transaction Verification



New Block Creation



Added to Blockchain Ledger

Simplified Prototype Design:

Step 1: Create Wallets

Two users are created:

- User A
- User B

Both have digital wallets.

Step 2: Create Digital Coin

A simple token named:

Coin Name: MyCoin

Initial Supply: 1000 coins

Step 3: Perform Transaction

User A sends 100 MyCoin to User B.

Step 4: Validate Transaction

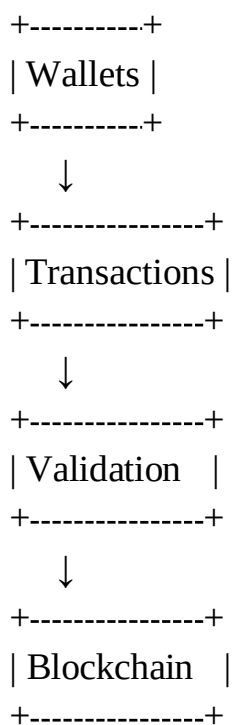
Blockchain platform verifies:

- Sufficient balance
 - Valid sender and receiver
-

Step 5: Add Block

After verification, transaction is stored in a new block.

Prototype Architecture:



Sample Code Concept:

- Smart contract written in Solidity (for Ethereum)
 - Token transfer function
 - Balance checking function
-

Applications:

- Digital payments
- Secure transactions
- Online trading
- Financial systems

Advantages:

- Transparency
- Security
- Fast transactions
- Decentralization

Conclusion:

Thus, a simplified cryptocurrency prototype was successfully developed using a blockchain development platform. It demonstrates the basic working of digital currency and blockchain transactions.

EXPERIMENT 07

Q1. Create simplified version of a Bitcoin wallet using software like Electrum.

ANS.

Aim:

To create a simplified version of a Bitcoin wallet using software like Electrum.

Introduction:

A Bitcoin wallet is a software application used to store, send, and receive Bitcoin. It contains public and private keys that help users perform secure transactions. Electrum is a lightweight and popular wallet used for managing Bitcoin transactions.

Requirements:

- Computer/Laptop
- Internet connection
- Electrum software
- Test Bitcoin network (optional)

Steps to Create Simplified Wallet:

Step 1: Install Wallet Software

Download and install Electrum on your system.

Step 2: Create New Wallet

- Open Electrum
- Select **Create New Wallet**
- Enter wallet name

Step 3: Generate Seed Phrase

Wallet generates a secret recovery phrase of 12 words.

Example:

apple mango tiger book river sun moon car tree fish pen house

This phrase helps recover wallet access.

Step 4: Create Password

Set a strong password to secure wallet access.

Step 5: Generate Bitcoin Address

Wallet creates a public address for receiving Bitcoin.

Example:

1A2B3C4D5E6F...

Wallet Working Diagram:

User → Wallet → Send/Receive Bitcoin → Blockchain Network

Sending Bitcoin:

1. Enter receiver wallet address
2. Enter amount
3. Confirm transaction

Receiving Bitcoin:

1. Share wallet address
2. Receive funds from sender

Security Features:

- Private key protection
- Seed phrase backup
- Password encryption

Applications:

- Online payments
- Investment
- Digital transactions

Advantages:

- Easy to use
- Secure transactions
- Fast transfers

Conclusion:

Thus, a simplified version of a Bitcoin wallet was created using Electrum, demonstrating basic wallet operations.

EXPERIMENT 08

Q1. Introduce the simplified version of Lamport -Shostak-Pease BFT algorithm for achieving Byzantine fault tolerance.

ANS.

Aim:

To study and introduce a simplified version of the **Lamport-Shostak-Pease Byzantine Fault Tolerance (BFT) Algorithm** for achieving fault tolerance in distributed systems.

Introduction:

The **Lamport-Shostak-Pease algorithm** was introduced in the famous The Byzantine Generals Problem by Leslie Lamport, Robert Shostak, and Marshall Pease.

It solves the **Byzantine Generals Problem**, where some nodes in a distributed network may fail or send false information. The algorithm helps honest nodes reach agreement even when some nodes are malicious.

In blockchain and distributed systems, BFT ensures reliability and security.

Problem Statement:

Suppose there are multiple generals (nodes) trying to agree on one decision:

- **Attack**
- **Retreat**

Some generals may act maliciously and send incorrect messages.

Goal: All honest generals must reach the same decision.

Basic Rule:

For **m faulty nodes**, the system requires:

$$n \geq 3m + 1$$

Where:

- n = total number of nodes
- m = faulty nodes

For 1 faulty node:

$$n \geq 3m + 1$$

Minimum nodes required = 4

Simplified Working Process:

Step 1: Commander Sends Message

One commander sends order to all lieutenants.

Example:

Commander → Attack

Step 2: Lieutenants Forward Message

Each lieutenant forwards received message to others.

L1 → L2

L2 → L3

L3 → L1

Step 3: Compare Messages

All nodes compare received messages.

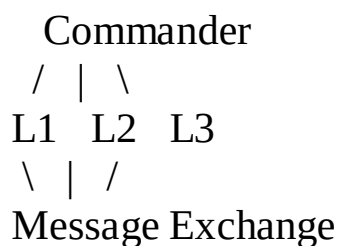
Step 4: Majority Decision

The final decision is based on majority voting.

Attack = 3 votes

Retreat = 1 vote

Final Decision = Attack

System Architecture Diagram:

Example:

If one lieutenant sends false information:

- L1 → Attack
- L2 → Attack
- L3 → Retreat (faulty)

Majority result:

Attack wins

Thus system remains fault tolerant.

Applications:

- Blockchain consensus
- Distributed databases
- Military communication systems
- Cloud computing

Advantages:

- Handles malicious nodes
- Ensures consensus
- Improves reliability

Limitations:

- High communication overhead
- Complex for large networks

Conclusion:

Thus, the simplified Lamport-Shostak-Pease BFT algorithm was introduced successfully. It helps achieve Byzantine fault tolerance in distributed systems by allowing honest nodes to agree even in the presence of faulty nodes.

EXPERIMENT 09

Q1. Implement a simplified version of blockchain solution for facilitating and tracking trade transactions.

ANS.

Aim:

To implement a simplified blockchain solution for facilitating and tracking trade transactions securely.

Introduction:

Blockchain is a distributed digital ledger that records transactions in blocks. Each block contains transaction details and is linked to the previous block using cryptographic hashes. In trade systems, blockchain helps track transactions between buyers, sellers, suppliers, and banks in a transparent and secure way.

This simplified model demonstrates how blockchain can be used for trade transaction tracking.

Requirements:

- Computer/Laptop
- Internet connection
- Python / Java
- Visual Studio Code
- Blockchain simulation platform

Participants in Trade Transaction:

1. Buyer
2. Seller
3. Supplier
4. Bank
5. Blockchain Network

Working Flow:

Buyer → Seller → Supplier → Bank



Transaction Data



Blockchain Ledger

Step-by-Step Implementation:

Step 1: Create Trade Request

Buyer places order for goods.

Example:

- Product: Electronics
- Quantity: 100 units

Step 2: Seller Accepts Order

Seller confirms order details.

Step 3: Payment Verification

Bank verifies payment transaction.

Step 4: Create Blockchain Block

A new block stores:

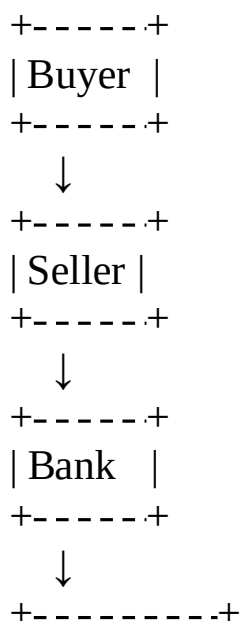
- Buyer details
- Seller details
- Product information
- Payment status

Step 5: Add Block to Chain

The block is added to blockchain ledger.

Block 1 → Block 2 → Block 3

Simplified Architecture Diagram:



| Blockchain |
+-----+

Benefits:

- Transparency
- Secure transaction records
- Easy tracking of goods
- Reduces fraud

Applications:

- Supply chain management
- International trade
- Banking transactions
- Logistics tracking

Advantages:

- Fast verification
- Immutable records
- Improved trust

Conclusion:

Thus, a simplified blockchain solution was successfully implemented for facilitating and tracking trade transactions. It improves transparency, security, and efficiency in trade systems.

EXPERIMENT 10

Q1. Implementing a simplified version of Ripple involves creating a basic blockchain with a consensus algorithm.

ANS.

Aim:

To implement a simplified version of XRP (Ripple Labs) by creating a basic blockchain system with a consensus algorithm.

Introduction:

Ripple Labs developed Ripple as a digital payment protocol that enables fast and low-cost international transactions. Unlike Bitcoin, Ripple does not use mining. It uses a **consensus algorithm** where trusted validators verify transactions.

This simplified prototype demonstrates how Ripple works using basic blockchain concepts.

Requirements:

- Computer/Laptop
- Python / Java
- Visual Studio Code
- Internet connection

Components:

1. Sender Account
2. Receiver Account
3. Validators
4. Transaction Ledger
5. Consensus Algorithm

Working Flow Diagram:

Sender → Validators → Consensus → Ledger → Receiver

Step-by-Step Process:

Step 1: Create Accounts

Two users are created:

- User A (Sender)
 - User B (Receiver)
-

Step 2: Initiate Transaction

User A sends money to User B.

Example:

A sends ₹500 to B

Step 3: Validators Verify Transaction

Multiple validators check:

- Sender balance
 - Transaction authenticity
 - Receiver account validity
-

Step 4: Consensus Process

Validators vote to approve transaction.

Validator 1 → Approve

Validator 2 → Approve

Validator 3 → Approve

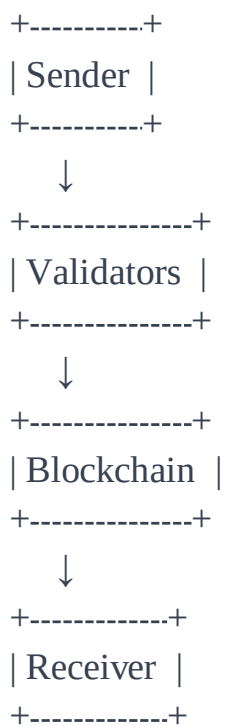
Majority approval confirms transaction.

Step 5: Update Ledger

After approval, transaction is added to blockchain ledger.

Block 1 → Block 2 → Block 3

Simplified Architecture:



Applications:

- International payments
- Banking transfers
- Cross-border transactions

Advantages:

- Fast transactions

- Low cost
- Energy efficient
- Secure validation

Conclusion:

Thus, a simplified version of Ripple Labs was implemented successfully using a blockchain ledger and consensus algorithm for secure transaction processing.